

Azure Data Engineering & Python

Deep-Dive Concepts • Code Examples • Practice Programs

ADF | Synapse | Databricks | PySpark | Snowflake | Python

Complete Technical Reference with Real-World Examples

SECTION 1: AZURE DATA FACTORY (ADF)

1. Azure Data Factory – In-Depth Guide

Azure Data Factory (ADF) is a cloud-based ETL, ELT and data-orchestration service. It lets you build, schedule and manage data pipelines that ingest, prepare, transform and publish data from hundreds of sources. Think of ADF as the 'plumbing' that moves data from where it lives to where it needs to be — reliably, at scale.

1.1 Architecture Overview

ADF has five building blocks that work together. Understanding each block is essential before writing a single pipeline.

Concept	Explanation
Pipeline	A logical container for a workflow of activities. A pipeline runs as a unit. You schedule or trigger the pipeline, not individual activities.
Activity	A single processing step inside a pipeline: Copy data, run a Spark job, call an API, evaluate a condition, iterate a list, etc.
Dataset	A named pointer to the data you want to work with — e.g., a specific table in SQL, a folder in ADLS, a topic in Event Hub.
Linked Service	The connection string (credentials + endpoint) for a data store or compute service. Stored in Azure Key Vault for security.
Integration Runtime (IR)	The physical compute that executes activities. Three flavours: Azure IR (cloud), Self-Hosted IR (on-premises), Azure-SSIS IR (legacy SSIS packages).

1.2 Linked Services — Deep Dive

A Linked Service is like a typed connection string. Every data store you connect to needs one. Best practice: always reference an Azure Key Vault linked service for credentials instead of storing them in plain text.

Example: ADLS Gen2 Linked Service (JSON definition)

JSON — linkedService_ADLS.json

```
{
  "name": "ls_adls_bronze",
  "type": "AzureDataLakeStorage",
  "typeProperties": {
    "url": "https://mystorageaccount.dfs.core.windows.net",
    "accountKey": {
      "type": "AzureKeyVaultSecret",
```

```

    "store": { "referenceName": "ls_keyvault", "type": "LinkedServiceReference"
  },
    "secretName": "adls-account-key"
  }
}

```

NOTE: Never store passwords or keys in plain text inside a Linked Service definition. Always use Azure Key Vault references.

1.3 Datasets — Deep Dive

A Dataset describes the structure, location, and format of data. Datasets are referenced by activities. They can be parameterised so that a single dataset definition covers thousands of files/tables.

Example: Parameterised Parquet Dataset

JSON — dataset_parquet_param.json

```

{
  "name": "ds_parquet_generic",
  "type": "Parquet",
  "linkedServiceName": { "referenceName": "ls_adls_bronze" },
  "parameters": {
    "containerName": { "type": "string" },
    "folderPath": { "type": "string" }
  },
  "typeProperties": {
    "location": {
      "type": "AzureDataLakeStorageLocation",
      "container": { "value": "@dataset().containerName" },
      "folderPath": { "value": "@dataset().folderPath" }
    },
    "compressionCodec": "snappy"
  }
}

```

1.4 Copy Activity — Hands-On

The Copy Activity is the workhorse of ADF. It moves data from a source connector to a sink connector. It supports schema mapping, data type conversion, fault tolerance, and high-throughput parallel copy.

Key Copy Activity Properties

- **source** — defines where to read data from (table, query, file pattern)
- **sink** — defines where to write data to (table, file, mode: overwrite/append/upsert)
- **parallelCopies** — number of concurrent threads; each thread copies a partition

- dataIntegrationUnits (DIU) — cloud compute power (2–256); higher = faster but costs more credits
- enableStaging — uses ADLS as intermediary; required for PolyBase bulk load into Synapse
- translator — explicit column mapping when source/sink schemas differ

Practice Program 1: Copy CSV → Parquet (ADF Pipeline JSON)

JSON — pipeline_csv_to_parquet.json

```
{
  "name": "pl_copy_csv_to_parquet",
  "properties": {
    "parameters": {
      "sourceContainer": { "type": "string", "defaultValue": "raw" },
      "sinkContainer": { "type": "string", "defaultValue": "bronze" }
    },
    "activities": [
      {
        "name": "CopyCsvToParquet",
        "type": "Copy",
        "inputs": [{ "referenceName": "ds_csv_source", "type": "DatasetReference" }],
        "outputs": [{ "referenceName": "ds_parquet_sink", "type": "DatasetReference" }],
        "typeProperties": {
          "source": {
            "type": "DelimitedTextSource",
            "storeSettings": { "type": "AzureDataLakeStoreReadSettings",
"recursive": true },
            "formatSettings": { "type": "DelimitedTextReadSettings" }
          },
          "sink": {
            "type": "ParquetSink",
            "storeSettings": { "type": "AzureDataLakeStoreWriteSettings" }
          },
          "parallelCopies": 8,
          "dataIntegrationUnits": 16,
          "enableSkipIncompatibleRow": true,
          "redirectIncompatibleRowSettings": {
            "linkedServiceName": "ls_adls_bronze",
            "path": "error-logs/@{pipeline().RunId}"
          }
        }
      }
    ]
  }
}
```

1.5 Incremental Load Pattern (Watermark)

Full loads are expensive. Incremental loads read only new/changed rows using a watermark — typically the last modified timestamp or an auto-increment ID stored in a control table.

Step-by-step Watermark Pattern

1. Store last_load_time in a control table (Azure SQL DB)
2. Lookup activity reads the current watermark
3. Copy Activity queries source WHERE modified_date > watermark
4. If Copy succeeds, update the watermark to the current run time

Control Table SQL

SQL — control_table.sql

```
-- Create watermark control table
CREATE TABLE dbo.WatermarkTable (
    TableName      NVARCHAR(128) NOT NULL PRIMARY KEY,
    WatermarkValue  DATETIME2     NOT NULL
);

-- Insert initial record
INSERT INTO dbo.WatermarkTable VALUES ('sales_orders', '1900-01-01');

-- Stored procedure to update watermark (called after successful copy)
CREATE PROCEDURE usp_UpdateWatermark
    @TableName      NVARCHAR(128),
    @NewWatermark    DATETIME2
AS
    UPDATE dbo.WatermarkTable
    SET     WatermarkValue = @NewWatermark
    WHERE  TableName      = @TableName;
```

ADF Expression — dynamic source query using watermark

ADF Expression Language

```
// In Copy Activity > Source > Query:
@concat(
    'SELECT * FROM dbo.SalesOrders WHERE ModifiedDate > ',
    string(activity('LookupWatermark').output.firstRow.WatermarkValue),
    ' AND ModifiedDate <= ',
    string(pipeline().TriggerTime),
    ''
)
```

1.6 Control Flow Activities

Control flow activities let you implement conditional logic, loops, waits, and sub-pipelines. They make ADF pipelines behave like real orchestration frameworks.

Concept	Explanation
If Condition	Evaluates a boolean expression and routes to 'true' or 'false' activity branches. Used for conditional routing.
ForEach	Iterates over an array and executes inner activities for each item. Set isSequential:false for parallel processing.

Until	Loops until a condition becomes true. Used for polling patterns (wait until a file appears).
Execute Pipeline	Calls another pipeline synchronously or asynchronously. Enables modular, reusable pipeline design.
Lookup	Reads a single row or a dataset from any ADF-supported source. Outputs can be used in subsequent expressions.
Get Metadata	Retrieves metadata: file size, last modified time, column list, item count. Used to validate before processing.
Set Variable	Sets a pipeline variable. Variables can hold strings, arrays, booleans — used for in-flight state tracking.
Append Variable	Appends an item to an array variable. Useful for collecting results from a ForEach loop.
Web Activity	Calls any REST API. Used to trigger external jobs, send alerts, or retrieve tokens.
Fail Activity	Explicitly fails the pipeline with a custom error message. Used for business rule validation.

Practice Program 2: ForEach + Execute Pipeline Pattern

JSON — ForEach over table list

```
// Lookup activity reads list of tables from config
// ForEach iterates and calls child pipeline for each table
{
  "name": "ForEachTable",
  "type": "ForEach",
  "dependsOn": [{ "activity": "LookupTableList", "dependencyConditions":
["Succeeded"] }],
  "typeProperties": {
    "isSequential": false,
    "batchCount": 10,
    "items": { "value": "@activity('LookupTableList').output.value", "type":
"Expression" },
    "activities": [
      {
        "name": "ExecuteChildPipeline",
        "type": "ExecutePipeline",
        "typeProperties": {
          "pipeline": { "referenceName": "pl_copy_single_table", "type":
"PipelineReference" },
          "parameters": {
            "tableName": { "value": "@item().TableName", "type": "Expression" },
            "schemaName": { "value": "@item().SchemaName", "type": "Expression" }
          },
          "waitOnCompletion": true
        }
      }
    ]
  }
}
```

1.7 Mapping Data Flows

Mapping Data Flows provide a visual, code-free transformation engine backed by Spark. Each transformation you add in the canvas is compiled to Spark and executed on a managed Spark cluster. No Spark knowledge required — but understanding the underlying execution helps you tune performance.

Important Data Flow Transformations

- Source — reads data; configure partition strategy for large datasets
- Sink — writes data; choose update method (insert, upsert, delete)
- Derived Column — add/replace columns with expressions (like SQL computed columns)
- Aggregate — GROUP BY with aggregations (sum, count, avg, max, min, stddev)
- Join — inner/left/right/full/cross/exists/not-exists joins
- Lookup — enrichment join returning first match from right stream
- Filter — filter rows by boolean expression
- Select — rename, drop, reorder columns; fix schema drift
- Conditional Split — route rows to different output streams based on conditions
- Union — stack multiple streams vertically
- Pivot/Unpivot — reshape data (rows to columns, columns to rows)
- Rank/Window — analytical window functions (row_number, lead, lag, cumulative sum)
- Flatten — explode arrays and nested structs from JSON/hierarchical data
- Parse — parse inline JSON strings into structured columns

Practice Program 3: Data Flow expression — SCD Type 1 upsert logic

Data Flow Expression — Derived Column

```
// Hash entire row for change detection
sha2(256, columns()) -- generates row hash; compare to existing to detect changes

// Business date logic in derived column
iif(isNull(order_date), currentDate(), toDate(order_date, 'yyyy-MM-dd'))

// Flag new vs existing rows
iif(isNull(existing_id), 'INSERT', iif(row_hash != existing_hash, 'UPDATE', 'NOCHANGE'))
```

1.8 Triggers

Schedule Trigger — run every weekday at 6AM

JSON — schedule_trigger.json

```
{
  "name": "trg_daily_6am",
  "type": "ScheduleTrigger",
  "typeProperties": {
    "recurrence": {
      "frequency": "Week",
      "interval": 1,
```

```

    "schedule": { "hours": [6], "minutes": [0], "weekDays":
["Monday", "Tuesday", "Wednesday", "Thursday", "Friday"] },
    "timeZone": "India Standard Time"
  }
},
"pipelines": [{ "pipelineReference": { "referenceName": "pl_daily_load" } }]
}

```

Tumbling Window Trigger — hourly with dependency

JSON — tumbling_window.json

```

{
  "name": "trg_hourly_window",
  "type": "TumblingWindowTrigger",
  "typeProperties": {
    "frequency": "Hour",
    "interval": 1,
    "startTime": "2024-01-01T00:00:00Z",
    "retryPolicy": { "count": 3, "intervalInSeconds": 30 },
    "maxConcurrency": 2
  },
  "pipeline": {
    "pipelineReference": { "referenceName": "pl_hourly_ingestion" },
    "parameters": {
      "windowStart": "@trigger().outputs.windowStartTime",
      "windowEnd": "@trigger().outputs.windowEndTime"
    }
  }
}

```

1.9 Error Handling & Alerts

Production ADF pipelines must handle failures gracefully. The recommended pattern is to attach 'On Failure' dependency paths to error-handling activities and notify via Logic Apps or Azure Monitor.

Error handling pattern — Set Variable + Web Activity to Teams

ADF Expression — error capture

```

// Set Variable 'errorMessage' on failure path:
@concat(
  'Pipeline: ', pipeline().Pipeline,
  ' | Activity: ', activity('CopyData').output.errors[0].Message,
  ' | Run ID: ', pipeline().RunId
)

// Web Activity body to post Teams alert:
@json(concat(
  '{"text":"ADF FAILURE: ',
  variables('errorMessage'),
  '"}'
))

```


1.10 Full Practice Programs

Practice Program 4: Full Incremental Pipeline (Python + ADF SDK)

Use the Azure SDK for Python to trigger ADF pipelines programmatically — useful for testing, CI/CD, and custom orchestration.

Python — trigger_adf_pipeline.py

```
from azure.identity import DefaultAzureCredential
from azure.mgmt.datafactory import DataFactoryManagementClient
from azure.mgmt.datafactory.models import CreateRunResponse
import time

SUBSCRIPTION_ID = 'your-subscription-id'
RESOURCE_GROUP = 'rg-dataplatform'
FACTORY_NAME = 'adf-myproject-dev'
PIPELINE_NAME = 'pl_incremental_load'

credential = DefaultAzureCredential()
client = DataFactoryManagementClient(credential, SUBSCRIPTION_ID)

# Trigger pipeline with parameters
run_response: CreateRunResponse = client.pipelines.create_run(
    RESOURCE_GROUP,
    FACTORY_NAME,
    PIPELINE_NAME,
    parameters={
        'sourceSchema': 'dbo',
        'targetContainer': 'bronze',
        'loadDate': '2024-01-15'
    }
)
run_id = run_response.run_id
print(f'Pipeline triggered. Run ID: {run_id}')

# Poll until pipeline finishes
while True:
    run = client.pipeline_runs.get(RESOURCE_GROUP, FACTORY_NAME, run_id)
    status = run.status
    print(f'Status: {status}')
    if status in ('Succeeded', 'Failed', 'Cancelled'):
        break
    time.sleep(15)

if status == 'Succeeded':
    print('Pipeline completed successfully!')
else:
    raise RuntimeError(f'Pipeline failed with status: {status}')
```

Practice Program 5: Generate ADF Pipeline Config Dynamically (Python)

Programmatically generate ADF pipeline JSON from a table list — useful for building metadata-driven frameworks.

Python — generate_adf_config.py

```

import json

tables = [
    {'schema': 'sales', 'table': 'Orders', 'key': 'order_id',
    'watermark_col': 'updated_at'},
    {'schema': 'hr', 'table': 'Employees', 'key': 'employee_id',
    'watermark_col': 'modified_date'},
    {'schema': 'finance', 'table': 'Invoices', 'key': 'invoice_id',
    'watermark_col': 'created_date'},
]

def build_copy_activity(table_config):
    return {
        'name': f"Copy_{table_config['schema']}_{table_config['table']}",
        'type': 'Copy',
        'typeProperties': {
            'source': {
                'type': 'SqlServerSource',
                'sqlReaderQuery': (
                    f"SELECT * FROM
{table_config['schema']}.{table_config['table']} "
                    f"WHERE {table_config['watermark_col']} >
'@{{pipeline().parameters.watermark}}'"
                )
            },
            'sink': {
                'type': 'ParquetSink',
                'storeSettings': {'type': 'AzureDataLakeStoreWriteSettings'}
            }
        }
    }

pipeline_def = {
    'name': 'pl_metadata_driven_copy',
    'properties': {
        'parameters': {'watermark': {'type': 'string'}},
        'activities': [build_copy_activity(t) for t in tables]
    }
}

with open('pipeline_generated.json', 'w') as f:
    json.dump(pipeline_def, f, indent=2)

print(f'Generated pipeline with {len(tables)} copy activities')

```

SECTION 2: AZURE SYNAPSE ANALYTICS

2. Azure Synapse Analytics – In-Depth Guide

Azure Synapse Analytics is an enterprise analytics service that unites data warehousing, big data, and data integration into a single platform. It is Microsoft's answer to modern data lakehouse architecture — combining the best of SQL data warehouses with the flexibility of Apache Spark over a shared Azure Data Lake.

2.1 Architecture Deep Dive

Concept	Explanation
Dedicated SQL Pool	Massively Parallel Processing (MPP) data warehouse. Provisions fixed DWUs. Best for heavy analytical queries, BI reports, and large joins.
Serverless SQL Pool	Query ADLS files (CSV, Parquet, Delta, JSON) with T-SQL on-demand. No infrastructure. Pay per TB scanned. Best for exploration and external tables.
Spark Pool	Apache Spark cluster (auto-scale, auto-pause). PySpark/Scala/SQL notebooks. Best for complex transformations, ML, and streaming.
Data Explorer Pool	High-performance time-series and log analytics using KQL. Best for IoT telemetry, application logs.
Synapse Pipelines	ADF-compatible orchestration within the workspace. Same connectors, triggers, and data flows as ADF.
Synapse Link	Zero-ETL analytics on operational databases (Cosmos DB, SQL DB, Dataverse). Near-real-time analytical replica.
Synapse Studio	Unified web IDE: develop notebooks and SQL scripts, build pipelines, monitor jobs, manage access — all in one browser tab.

2.2 MPP Architecture — How Queries Execute

Understanding how Synapse executes a query helps you design tables correctly and avoid slow performance.

5. Client submits SQL query to the Control Node
6. Control Node parses, optimises and creates a Distributed Query Plan (DQP)
7. DQP is split into parallel operations called 'steps'
8. Each step is sent to the 60 Compute Nodes
9. Compute Nodes execute against their local data partitions (60 distributions each)
10. Data Movement Service (DMS) shuffles data across nodes when required (JOIN, GROUP BY)

11. Results are aggregated back to the Control Node and returned to client

NOTE: The goal of good table design is to minimise Data Movement (DMS). Every shuffle is expensive — it writes data to disk and transfers over the network. Hash distributing tables on the JOIN key eliminates DMS for that join.

2.3 Table Distribution — Detailed

Hash Distribution

Rows are assigned to one of the 60 distributions based on a hash of the distribution column. All rows with the same key value land on the same node — enabling co-located joins with no DMS.

SQL — Hash-distributed fact table

```
CREATE TABLE dbo.FactSales
WITH (
    DISTRIBUTION = HASH(customer_id),    -- must be high-cardinality, used in JOINS
    CLUSTERED COLUMNSTORE INDEX,
    PARTITION (order_date RANGE RIGHT FOR VALUES ('2022-01-01','2023-01-01','2024-01-01'))
)
AS
SELECT
    order_id, customer_id, product_id,
    order_date, quantity, unit_price,
    quantity * unit_price AS revenue
FROM staging.SalesOrders;
```

Round Robin Distribution

SQL — Round Robin staging table

```
-- Good for staging: fast loads, no skew risk
CREATE TABLE stg.SalesOrders
WITH ( DISTRIBUTION = ROUND_ROBIN, HEAP )    -- HEAP = no index, fastest for bulk loads
AS SELECT * FROM OPENROWSET(
    BULK 'https://adls.dfs.core.windows.net/raw/sales/*.parquet',
    FORMAT = 'PARQUET'
) AS src;
```

Replicated Distribution

SQL — Replicated dimension table

```
-- Good for small dimensions (<2GB) that are frequently JOINed
CREATE TABLE dbo.DimProduct
WITH ( DISTRIBUTION = REPLICATE, CLUSTERED INDEX (product_id) )
AS
SELECT product_id, product_name, category, brand, list_price
FROM stg.Products;
```

2.4 Table Types and Indexes

Clustered Columnstore Index (CCI) — Default for Analytics

CCI stores data in column groups called row groups (up to 1,048,576 rows each). The query engine reads only the columns needed and uses vectorised execution. Compression is typically 10x vs row store.

SQL — Rebuilding a fragmented CCI

```
-- Check CCI health
SELECT  object_name(object_id)          AS table_name,
        row_group_id,
        state_description,
        total_rows,
        deleted_rows,
        size_in_bytes / 1024 / 1024    AS size_mb
FROM    sys.dm_pdw_nodes_db_column_store_row_group_physical_stats
WHERE   object_id = OBJECT_ID('dbo.FactSales');

-- Rebuild to consolidate OPEN/CLOSED small row groups
ALTER INDEX ALL ON dbo.FactSales REBUILD;
```

2.5 Statistics — Critical for Performance

The query optimiser uses column statistics to estimate cardinality and choose the optimal join and aggregation algorithms. Stale or missing statistics cause bad execution plans.

SQL — Creating and updating statistics

```
-- Create statistics on join and filter columns
CREATE STATISTICS stat_customer_id ON dbo.FactSales (customer_id);
CREATE STATISTICS stat_order_date  ON dbo.FactSales (order_date);
CREATE STATISTICS stat_product_id  ON dbo.FactSales (product_id);

-- Update after significant data load (>20% row change)
UPDATE STATISTICS dbo.FactSales;

-- Check statistics age
SELECT  OBJECT_NAME(s.object_id)  AS table_name,
        c.name                    AS column_name,
        s.name                    AS stat_name,
        STATS_DATE(s.object_id, s.stats_id) AS last_updated
FROM    sys.stats                s
JOIN    sys.stats_columns sc ON s.object_id = sc.object_id AND s.stats_id =
sc.stats_id
JOIN    sys.columns      c  ON sc.object_id = c.object_id AND sc.column_id =
c.column_id
WHERE   OBJECT_NAME(s.object_id) = 'FactSales'
ORDER BY last_updated DESC;
```

2.6 Serverless SQL Pool — Query Data Lake

Serverless SQL Pool lets you query files in Azure Data Lake Storage Gen2 using familiar T-SQL syntax — no loading required. Perfect for exploratory analysis and building external tables for reports.

Practice Program 6: Query Parquet files with OPENROWSET

SQL — serverless_query.sql

```
-- Query a single Parquet file
SELECT TOP 100 *
FROM OPENROWSET(
    BULK 'https://adls.dfs.core.windows.net/bronze/sales/2024/**',
    FORMAT = 'PARQUET'
) AS r;

-- Query with explicit schema
SELECT order_id, customer_id, CAST(order_date AS DATE) AS order_date, amount
FROM OPENROWSET(
    BULK 'https://adls.dfs.core.windows.net/bronze/sales/2024/**',
    FORMAT = 'PARQUET'
) WITH (
    order_id      BIGINT,
    customer_id   INT,
    order_date     NVARCHAR(20),
    amount        DECIMAL(18,2)
) AS r
WHERE CAST(order_date AS DATE) BETWEEN '2024-01-01' AND '2024-03-31';

-- Create persistent external table
CREATE EXTERNAL DATA SOURCE adls_bronze
WITH ( LOCATION = 'https://adls.dfs.core.windows.net/bronze' );

CREATE EXTERNAL FILE FORMAT fmt_parquet
WITH ( FORMAT_TYPE = PARQUET, DATA_COMPRESSION =
'org.apache.hadoop.io.compress.SnappyCodec' );

CREATE EXTERNAL TABLE ext_sales
WITH (
    DATA_SOURCE = adls_bronze,
    LOCATION = 'sales/2024/',
    FILE_FORMAT = fmt_parquet
)
AS
SELECT order_id, customer_id, order_date, amount
FROM OPENROWSET(BULK 'sales/2024/**', DATA_SOURCE='adls_bronze', FORMAT='PARQUET')
AS r;
```

2.7 Workload Management

In a multi-user environment, you must prevent one heavy query from starving others. Workload Management uses resource groups and classifiers to allocate DWUs fairly.

SQL — workload_management.sql

```
-- Create workload groups
CREATE WORKLOAD GROUP wg_etl_critical
WITH (
    MIN_PERCENTAGE_RESOURCE = 40,      -- ETL always gets at least 40% of DWU
```

```

CAP_PERCENTAGE_RESOURCE = 80,      -- ETL can use at most 80%
REQUEST_MIN_RESOURCE_GRANT_PERCENT = 10  -- each request gets min 10%
);

CREATE WORKLOAD GROUP wg_analyst
WITH (
    MIN_PERCENTAGE_RESOURCE = 10,
    CAP_PERCENTAGE_RESOURCE = 30,
    REQUEST_MIN_RESOURCE_GRANT_PERCENT = 2
);

-- Create workload classifiers
CREATE WORKLOAD CLASSIFIER cls_etl_user
WITH (
    WORKLOAD_GROUP = 'wg_etl_critical',
    MEMBERNAME = 'etl_service_account',
    IMPORTANCE = HIGH
);

CREATE WORKLOAD CLASSIFIER cls_analyst
WITH (
    WORKLOAD_GROUP = 'wg_analyst',
    MEMBERNAME = 'analyst_role',
    IMPORTANCE = NORMAL
);

```

2.8 Full Practice Programs — Synapse

Practice Program 7: Star Schema Build with CTAS

SQL — build_star_schema.sql

```

-- Step 1: Load staging table from Data Lake
CREATE TABLE stg.RawSales
WITH ( DISTRIBUTION = ROUND_ROBIN, HEAP )
AS
SELECT * FROM OPENROWSET(
    BULK 'https://adls.dfs.core.windows.net/raw/sales/2024/**',
    FORMAT = 'PARQUET'
) AS r;

-- Step 2: Build dimension
CREATE TABLE dbo.DimCustomer
WITH ( DISTRIBUTION = REPLICATE, CLUSTERED INDEX(customer_key) )
AS
SELECT
    ROW_NUMBER() OVER (ORDER BY customer_id) AS customer_key,
    customer_id, first_name, last_name, email,
    city, country, segment
FROM (
    SELECT DISTINCT customer_id, first_name, last_name,
                    email, city, country, segment
    FROM stg.RawSales
) t;

-- Step 3: Build fact table
CREATE TABLE dbo.FactSales
WITH (

```

```

        DISTRIBUTION = HASH(customer_key),
        CLUSTERED COLUMNSTORE INDEX,
        PARTITION (order_date RANGE RIGHT FOR VALUES ('2023-01-01','2024-01-01'))
    )
AS
SELECT
    s.order_id,
    c.customer_key,
    s.product_id,
    CAST(s.order_date AS DATE)      AS order_date,
    s.quantity,
    s.unit_price,
    s.quantity * s.unit_price      AS revenue,
    GETDATE()                      AS etl_loaded_at
FROM stg.RawSales s
JOIN dbo.DimCustomer c ON s.customer_id = c.customer_id;

-- Step 4: Create statistics
CREATE STATISTICS stat_cust_key ON dbo.FactSales (customer_key);
CREATE STATISTICS stat_order_dt ON dbo.FactSales (order_date);
CREATE STATISTICS stat_prod_id ON dbo.FactSales (product_id);

```

Practice Program 8: Python script to connect to Synapse Dedicated Pool

Python — synapse_connector.py

```

import pyodbc
import pandas as pd
from azure.identity import DefaultAzureCredential
import struct

SERVER    = 'mysynapse.sql.azuresynapse.net'
DATABASE = 'mySynapseDB'

# Use Azure AD token (no password needed with Managed Identity)
credential = DefaultAzureCredential()
token = credential.get_token('https://database.windows.net/.default').token
token_bytes = bytes(token, 'utf-8')
expiry = struct.pack('<I', 0)
token_struct = struct.pack(f'<I{len(token_bytes)}s', len(token_bytes), token_bytes)

conn_str = (
    f'DRIVER={{ODBC Driver 18 for SQL Server}};'
    f'SERVER={SERVER};DATABASE={DATABASE};'
    'Authentication=ActiveDirectoryServicePrincipal;'
)

conn = pyodbc.connect(conn_str, attrs_before={1256: token_struct})

# Run query and return DataFrame
query = '''
    SELECT  c.country,
            SUM(f.revenue) AS total_revenue,
            COUNT(DISTINCT f.customer_key) AS unique_customers
    FROM    dbo.FactSales f
    JOIN    dbo.DimCustomer c ON f.customer_key = c.customer_key
    WHERE   f.order_date >= '2024-01-01'
    GROUP BY c.country
'''

```



```
        ORDER BY total_revenue DESC
'''
df = pd.read_sql(query, conn)
print(df.head(10))
conn.close()
```

SECTION 3: AZURE DATABRICKS

3. Azure Databricks – In-Depth Guide

Azure Databricks is an Apache Spark-based analytics platform engineered for speed and collaboration. It integrates deeply with Azure services, provides enterprise-grade security via Azure Active Directory, and makes Delta Lake the default storage format — enabling ACID transactions directly on your Data Lake.

3.1 Cluster Architecture

Concept	Explanation
Driver Node	JVM process that hosts the SparkContext, coordinates task scheduling, and aggregates results. Runs notebooks interactively. If the driver OOMs, the cluster dies.
Worker Nodes	Execute Spark Tasks in parallel. Each worker has one or more executor JVM processes. Scale out by adding more workers.
Executor	JVM process on a worker that runs tasks. Has memory (Execution + Storage pools) and CPU slots (= number of cores).
Task	Smallest unit of work: processes one partition of data. Runs inside an executor core. 1 partition = 1 task.
Slot / Core	One CPU core in an executor = one slot that can run one task at a time. Total parallelism = total slots across all executors.

3.2 Delta Lake — Complete Guide

Delta Lake is the storage foundation for all Databricks workloads. It adds a transaction log (`_delta_log` directory) alongside your Parquet files. Every write operation appends a JSON entry to the log, giving you ACID guarantees, versioning, and metadata statistics.

Delta Lake Transaction Log

- Each commit creates a numbered JSON file in `_delta_log/` (e.g., `00000000000000000001.json`)
- Every 10 commits, a Parquet checkpoint is written to consolidate the log
- The log records the full state: which files are added/removed, schema, statistics
- Time Travel reconstructs historical state by replaying log entries up to a version

Practice Program 9: Full Delta Lake Operations

PySpark — `delta_lake_operations.py`

```

from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from delta.tables import DeltaTable

spark = SparkSession.builder \
    .appName('DeltaLakeOps') \
    .config('spark.sql.extensions', 'io.delta.sql.DeltaSparkSessionExtension') \
    .config('spark.sql.catalog.spark_catalog',
'org.apache.spark.sql.delta.catalog.DeltaCatalog') \
    .getOrCreate()

DELTA_PATH = '/mnt/bronze/delta/customers'

# — Step 1: Create initial Delta table
customers = spark.createDataFrame([
    (1, 'Alice', 'alice@example.com', 'Gold', '2024-01-01'),
    (2, 'Bob', 'bob@example.com', 'Silver', '2024-01-01'),
    (3, 'Carol', 'carol@example.com', 'Gold', '2024-01-01'),
], ['id', 'name', 'email', 'tier', 'updated_at'])

customers.write.format('delta').mode('overwrite').save(DELTA_PATH)
print('Version 0 created')

# — Step 2: Append new rows
new_customers = spark.createDataFrame([
    (4, 'Dave', 'dave@example.com', 'Bronze', '2024-01-15'),
    (5, 'Eve', 'eve@example.com', 'Gold', '2024-01-15'),
], ['id', 'name', 'email', 'tier', 'updated_at'])

new_customers.write.format('delta').mode('append').save(DELTA_PATH)
print('Version 1: appended 2 rows')

# — Step 3: MERGE (upsert) — SCD Type 1
updates = spark.createDataFrame([
    (2, 'Bob Updated', 'bob_new@example.com', 'Gold', '2024-02-01'), # update
    (6, 'Frank', 'frank@example.com', 'Silver', '2024-02-01'), # insert
], ['id', 'name', 'email', 'tier', 'updated_at'])

dt = DeltaTable.forPath(spark, DELTA_PATH)
dt.alias('tgt').merge(
    updates.alias('src'),
    'tgt.id = src.id'
).whenMatchedUpdateAll() \
    .whenNotMatchedInsertAll() \
    .execute()
print('Version 2: merged updates')

# — Step 4: Delete rows
dt.delete(F.col('tier') == 'Bronze')
print('Version 3: deleted Bronze tier')

# — Step 5: View history
dt.history().select('version', 'timestamp', 'operation', 'operationMetrics').show(truncate=False)

# — Step 6: Time Travel — read version 0
v0 = spark.read.format('delta').option('versionAsOf', 0).load(DELTA_PATH)
print('Version 0 rows:', v0.count())

```

```
# — Step 7: Time Travel – read by timestamp
v_ts = spark.read.format('delta') \
    .option('timestampAsOf', '2024-01-15 00:00:00') \
    .load(DELTA_PATH)
print('Rows at Jan 15:', v_ts.count())

# — Step 8: OPTIMIZE + ZORDER
spark.sql(f"OPTIMIZE delta.`{DELTA_PATH}` ZORDER BY (id)")
print('Compacted small files and Z-ordered')

# — Step 9: VACUUM (remove old files)
spark.conf.set('spark.databricks.delta.retentionDurationCheck.enabled', 'false')
spark.sql(f"VACUUM delta.`{DELTA_PATH}` RETAIN 168 HOURS")
print('Vacuum complete')
```

Practice Program 10: SCD Type 2 with Delta MERGE

SCD Type 2 preserves historical records by closing the old row and inserting a new one when an attribute changes.

PySpark — scd_type2.py

```
from pyspark.sql import functions as F
from delta.tables import DeltaTable

# Current dimension table (Delta)
DIM_PATH = '/mnt/gold/dim_customer'

# Incoming CDC feed (changed rows only)
cdc = spark.createDataFrame([
    (101, 'Alice Smith', 'alice@example.com', 'Platinum', '2024-03-01'),
    (102, 'Bob Jones', 'bob_new@corp.com', 'Gold', '2024-03-01'),
], ['customer_id', 'name', 'email', 'tier', 'effective_date'])

dim = DeltaTable.forPath(spark, DIM_PATH)

# Step 1: Mark existing active rows for closure
dim.alias('dim').merge(
    cdc.alias('src'),
    'dim.customer_id = src.customer_id AND dim.is_current = true'
).whenMatchedUpdate(
    condition = 'dim.tier != src.tier OR dim.email != src.email',
    set = {
        'is_current': F.lit(False),
        'end_date': F.col('src.effective_date'),
        'updated_at': F.current_timestamp()
    }
).execute()

# Step 2: Insert new active rows
new_rows = cdc \
    .withColumn('is_current', F.lit(True)) \
    .withColumn('start_date', F.col('effective_date')) \
    .withColumn('end_date', F.lit('9999-12-31')) \
    .withColumn('updated_at', F.current_timestamp())

new_rows.write.format('delta').mode('append').save(DIM_PATH)
print('SCD Type 2 merge complete')
```

3.3 Databricks Autoloader

Autoloader is the recommended way to ingest files from cloud storage into Delta Lake incrementally. It tracks processed files using a checkpoint, handles schema inference, and scales automatically.

Practice Program 11: Autoloader Streaming Ingestion

PySpark — autoloader_ingest.py

```
from pyspark.sql import functions as F
from pyspark.sql.types import StructType, StructField, StringType, DoubleType,
TimestampType

LANDING_PATH    = 'abfss://landing@adls.dfs.core.windows.net/events/'
BRONZE_PATH     = 'abfss://bronze@adls.dfs.core.windows.net/delta/events/'
CHECKPOINT_PATH = 'abfss://checkpoints@adls.dfs.core.windows.net/autoloader/events/'

# Explicit schema (preferred over inferSchema for production)
schema = StructType([
    StructField('event_id',    StringType(),    False),
    StructField('user_id',     StringType(),    True),
    StructField('event_type',  StringType(),    True),
    StructField('amount',      DoubleType(),    True),
    StructField('event_time',  TimestampType(), True),
])

# Autoloader read stream
stream_df = spark.readStream \
    .format('cloudFiles') \
    .option('cloudFiles.format', 'json') \
    .option('cloudFiles.schemaLocation', CHECKPOINT_PATH + 'schema') \
    .option('cloudFiles.inferColumnTypes', 'true') \
    .option('cloudFiles.schemaEvolutionMode', 'addNewColumns') \
    .schema(schema) \
    .load(LANDING_PATH)

# Add ingestion metadata
enriched = stream_df \
    .withColumn('ingestion_time', F.current_timestamp()) \
    .withColumn('source_file', F.input_file_name()) \
    .withColumn('year', F.year('event_time')) \
    .withColumn('month', F.month('event_time')) \
    .withColumn('day', F.dayofmonth('event_time'))

# Write to Delta Lake with micro-batch trigger
query = enriched.writeStream \
    .format('delta') \
    .outputMode('append') \
    .option('checkpointLocation', CHECKPOINT_PATH) \
    .option('mergeSchema', 'true') \
    .partitionBy('year', 'month', 'day') \
    .trigger(availableNow=True) \
    .start(BRONZE_PATH)
```

```
query.awaitTermination()
print(f'Ingested in {query.lastProgress["numInputRows"]} rows')
```

3.4 Medallion Architecture — Full Example

The Medallion Architecture organises a Data Lakehouse into three progressive quality layers: Bronze (raw), Silver (cleaned), Gold (aggregated). Each layer uses Delta Lake for ACID guarantees.

Practice Program 12: Bronze → Silver → Gold Pipeline

PySpark — medallion_pipeline.py

```
from pyspark.sql import functions as F, SparkSession
from delta.tables import DeltaTable

spark = SparkSession.builder.appName('Medallion').getOrCreate()

BRONZE = '/mnt/bronze/delta/orders'
SILVER = '/mnt/silver/delta/orders_clean'
GOLD   = '/mnt/gold/delta/orders_agg'

# === BRONZE: raw ingest ===
raw = spark.read.format('csv') \
    .option('header', 'true') \
    .option('inferSchema', 'true') \
    .load('/mnt/landing/orders/*.csv')

raw.withColumn('_ingest_time', F.current_timestamp()) \
    .withColumn('_source_file', F.input_file_name()) \
    .write.format('delta').mode('append').save(BRONZE)
print(f'Bronze: {raw.count()} rows ingested')

# === SILVER: clean and validate ===
bronze_df = spark.read.format('delta').load(BRONZE)

silver_df = bronze_df \
    .dropDuplicates(['order_id']) \
    .filter(F.col('order_id').isNotNull()) \
    .filter(F.col('amount') > 0) \
    .withColumn('order_date', F.to_date('order_date', 'yyyy-MM-dd')) \
    .withColumn('amount', F.col('amount').cast('double')) \
    .withColumn('customer_id', F.col('customer_id').cast('integer')) \
    .withColumn('status', F.upper(F.trim('status'))) \
    .withColumn('is_large', F.col('amount') > 1000) \
    .select('order_id', 'customer_id', 'order_date', 'amount', 'status', 'is_large')

# Upsert into Silver Delta (idempotent)
if DeltaTable.isDeltaTable(spark, SILVER):
    dt = DeltaTable.forPath(spark, SILVER)
    dt.alias('t').merge(silver_df.alias('s'), 't.order_id = s.order_id') \
        .whenMatchedUpdateAll() \
        .whenNotMatchedInsertAll() \
        .execute()
else:
    silver_df.write.format('delta').partitionBy('order_date').save(SILVER)
```

```

print('Silver layer updated')

# === GOLD: business aggregates ===
silver = spark.read.format('delta').load(SILVER)

gold_df = silver \
    .filter(F.col('status') == 'COMPLETED') \
    .groupBy(
        F.year('order_date').alias('year'),
        F.month('order_date').alias('month'),
        'customer_id'
    ).agg(
        F.count('order_id').alias('order_count'),
        F.sum('amount').alias('total_revenue'),
        F.avg('amount').alias('avg_order_value'),
        F.max('amount').alias('max_order'),
        F.countDistinct('order_date').alias('active_days')
    ) \
    .withColumn('revenue_rank',
        F.dense_rank().over(
            __import__('pyspark.sql.window', fromlist=['Window']).Window
            .partitionBy('year', 'month').orderBy(F.desc('total_revenue'))
        )
    )

gold_df.write.format('delta').mode('overwrite').partitionBy('year', 'month').save(GOLD)
print('Gold layer refreshed')
gold_df.orderBy('year', 'month', 'revenue_rank').show(20)

```

3.5 Unity Catalog — Governance

Practice Program 13: Unity Catalog DDL

SQL — unity_catalog_setup.sql

```

-- Create catalog (top-level namespace)
CREATE CATALOG IF NOT EXISTS prod_catalog
    COMMENT 'Production data assets';

-- Create schemas (databases)
CREATE SCHEMA IF NOT EXISTS prod_catalog.bronze COMMENT 'Raw ingested data';
CREATE SCHEMA IF NOT EXISTS prod_catalog.silver COMMENT 'Cleaned data';
CREATE SCHEMA IF NOT EXISTS prod_catalog.gold    COMMENT 'Business aggregates';

-- Register Delta table in Unity Catalog
CREATE TABLE prod_catalog.silver.orders
    USING DELTA
    LOCATION 'abfss://silver@adls.dfs.core.windows.net/delta/orders'
    COMMENT 'Cleaned order transactions';

-- Grant access
GRANT USE CATALOG ON CATALOG prod_catalog TO `data-engineers`;
GRANT USE SCHEMA  ON SCHEMA prod_catalog.gold TO `data-analysts`;
GRANT SELECT      ON TABLE prod_catalog.gold.orders_agg TO `data-analysts`;

-- Column-level masking (mask email for non-privileged users)
CREATE FUNCTION prod_catalog.security.mask_email(email STRING)

```

```
RETURNS STRING
RETURN CASE WHEN IS_MEMBER('data-engineers') THEN email
              ELSE CONCAT(LEFT(email,2), '***@***') END;

ALTER TABLE prod_catalog.silver.customers
  ALTER COLUMN email SET MASK prod_catalog.security.mask_email;
```


SECTION 4: PYSPARK — DEEP DIVE

4. PySpark – In-Depth Guide

PySpark is the Python API for Apache Spark. Spark executes computation across a cluster of machines using a DAG (Directed Acyclic Graph) of stages and tasks. Understanding the execution model is key to writing efficient PySpark code.

4.1 Spark Execution Model

Concept	Explanation
Job	One Spark action (collect, count, write) triggers one Job.
Stage	A Job is split into Stages at shuffle boundaries (groupBy, join, sort). All tasks in a Stage can run without shuffling.
Task	Each Stage is split into Tasks — one task per partition. Tasks run in parallel across Executor cores.
Shuffle	Data redistribution across partitions/nodes. Triggered by wide transformations. Written to disk, transferred over network.
Partition	Logical division of a DataFrame. Processed by one Task. More partitions = more parallelism, but too many = scheduling overhead.

4.2 DataFrame API — Complete Operations

Practice Program 14: All Essential DataFrame Operations

```
PySpark — dataframe_operations.py
from pyspark.sql import SparkSession, functions as F, Window
from pyspark.sql.types import *

spark = SparkSession.builder.appName('DataFrameOps').getOrCreate()

# — Schema Definition —
schema = StructType([
    StructField('order_id', IntegerType(), False),
    StructField('customer_id', IntegerType(), True),
    StructField('product', StringType(), True),
    StructField('category', StringType(), True),
    StructField('amount', DoubleType(), True),
    StructField('order_date', StringType(), True),
    StructField('region', StringType(), True),
])

data = [
    (1, 101, 'Laptop', 'Electronics', 1200.0, '2024-01-05', 'North'),
    (2, 102, 'Phone', 'Electronics', 800.0, '2024-01-10', 'South'),
```

```

    (3, 101, 'Tablet', 'Electronics', 500.0, '2024-01-15', 'North'),
    (4, 103, 'Chair', 'Furniture', 300.0, '2024-02-01', 'East'),
    (5, 102, 'Desk', 'Furniture', 700.0, '2024-02-10', 'South'),
    (6, 104, 'Monitor', 'Electronics', 450.0, '2024-02-15', 'West'),
    (7, 103, 'Lamp', 'Furniture', 80.0, '2024-03-01', 'East'),
    (8, 105, 'Phone', 'Electronics', 820.0, '2024-03-10', None),
]
df = spark.createDataFrame(data, schema)

# — Basic Ops —————
print('=== Schema ===')
df.printSchema()

print('=== Select & Filter ===')
df.select('order_id', 'customer_id', 'amount') \
    .filter((F.col('amount') > 500) & F.col('region').isNotNull()) \
    .show()

# — Column Expressions —————
print('=== Derived Columns ===')
df2 = df \
    .withColumn('order_date', F.to_date('order_date', 'yyyy-MM-dd')) \
    .withColumn('year', F.year('order_date')) \
    .withColumn('month', F.month('order_date')) \
    .withColumn('quarter', F.quarter('order_date')) \
    .withColumn('amount_eur', F.round(F.col('amount') * 0.92, 2)) \
    .withColumn('size_class', F.when(F.col('amount') >= 1000, 'Large')
        .when(F.col('amount') >= 500, 'Medium')
        .otherwise('Small')) \
    .withColumn('region', F.coalesce('region', F.lit('Unknown')))
df2.show()

# — Aggregations —————
print('=== Aggregations ===')
agg_df = df2.groupBy('category', 'region').agg(
    F.count('order_id').alias('order_count'),
    F.sum('amount').alias('total_revenue'),
    F.avg('amount').alias('avg_revenue'),
    F.max('amount').alias('max_revenue'),
    F.min('amount').alias('min_revenue'),
    F.countDistinct('customer_id').alias('unique_customers'),
    F.collect_list('product').alias('products_sold')
)
agg_df.orderBy('total_revenue', ascending=False).show(truncate=False)

# — Window Functions —————
print('=== Window Functions ===')
w_cat = Window.partitionBy('category').orderBy(F.desc('amount'))
w_cust = Window.partitionBy('customer_id').orderBy('order_date')
w_cum = Window.partitionBy('customer_id').orderBy('order_date') \
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)

windowed = df2 \
    .withColumn('rank_in_category', F.rank().over(w_cat)) \
    .withColumn('dense_rank', F.dense_rank().over(w_cat)) \
    .withColumn('row_num', F.row_number().over(w_cat)) \
    .withColumn('prev_amount', F.lag('amount', 1).over(w_cust)) \
    .withColumn('next_amount', F.lead('amount', 1).over(w_cust)) \
    .withColumn('running_revenue', F.sum('amount').over(w_cum)) \
    .withColumn('pct_of_category', F.round(

```

```

        F.col('amount') / F.sum('amount').over(Window.partitionBy('category')) *
100, 1
    ))

windowed.select(
    'customer_id','product','category','amount',
    'rank_in_category','running_revenue','pct_of_category'
).show()

```

4.3 Joins — All Types

Practice Program 15: Join Operations

PySpark — joins.py

```

# Customers dimension
customers = spark.createDataFrame([
    (101, 'Alice', 'Premium'), (102, 'Bob', 'Standard'),
    (103, 'Carol', 'Premium'), (106, 'Frank', 'Standard')
], ['customer_id', 'name', 'tier'])

orders = spark.createDataFrame([
    (1, 101, 500.0), (2, 102, 300.0), (3, 101, 800.0),
    (4, 104, 200.0), (5, 105, 150.0) # 104,105 have no customer record
], ['order_id', 'customer_id', 'amount'])

# Inner join — only matching rows
orders.join(customers, 'customer_id', 'inner').show()

# Left join — all orders, null for unmatched customers
orders.join(customers, 'customer_id', 'left').show()

# Left anti join — orders WITHOUT a customer record
orders.join(customers, 'customer_id', 'left_anti').show()

# Left semi join — orders THAT HAVE a customer record (no customer cols)
orders.join(customers, 'customer_id', 'left_semi').show()

# Cross join (be careful — cartesian product!)
small_a = spark.createDataFrame([('X',), ('Y',)], ['label'])
small_b = spark.createDataFrame([(1,), (2,)], ['value'])
small_a.crossJoin(small_b).show()

# Broadcast join — force small DF to be broadcast
from pyspark.sql.functions import broadcast
orders.join(broadcast(customers), 'customer_id').show()

# Non-equi join — amount range
tiers = spark.createDataFrame([
    ('Small', 0, 299),
    ('Medium', 300, 699),
    ('Large', 700, 9999)
], ['tier_name', 'min_amount', 'max_amount'])

orders.alias('o').join(
    tiers.alias('t'),
    (F.col('o.amount') >= F.col('t.min_amount')) &

```

```
(F.col('o.amount') <= F.col('t.max_amount')),
    'inner'
).show()
```

4.4 User Defined Functions (UDF & Pandas UDF)

Practice Program 16: UDFs

PySpark — udfs.py

```
from pyspark.sql.functions import udf, pandas_udf
from pyspark.sql.types import StringType, DoubleType
import pandas as pd
import re

# — Regular UDF (row-by-row, slow for large datasets) —
@udf(returnType=StringType())
def clean_phone(phone: str) -> str:
    '''Remove non-digit characters from phone number'''
    if phone is None:
        return None
    digits = re.sub(r'\D', '', phone)
    return digits if len(digits) == 10 else None

# — Pandas UDF (vectorised via Apache Arrow) —
# ~10x faster than regular UDF for numeric operations
@pandas_udf(DoubleType())
def haversine_distance(lat1: pd.Series, lon1: pd.Series,
                       lat2: pd.Series, lon2: pd.Series) -> pd.Series:
    '''Calculate great-circle distance in km between two points'''
    import numpy as np
    R = 6371.0
    phi1, phi2 = np.radians(lat1), np.radians(lat2)
    dphi = np.radians(lat2 - lat1)
    dlamba = np.radians(lon2 - lon1)
    a = np.sin(dphi/2)**2 + np.cos(phi1) * np.cos(phi2) * np.sin(dlamba/2)**2
    return R * 2 * np.arctan2(np.sqrt(a), np.sqrt(1 - a))

# Usage
locations = spark.createDataFrame([
    ('Store A', 28.6139, 77.2090, 28.5355, 77.3910),
    ('Store B', 19.0760, 72.8777, 18.9220, 72.8347),
], ['name', 'lat1', 'lon1', 'lat2', 'lon2'])

locations.withColumn(
    'distance_km',
    haversine_distance('lat1', 'lon1', 'lat2', 'lon2')
).show()
```

4.5 Structured Streaming

Practice Program 17: Real-time Aggregation with Watermark

PySpark — structured_streaming.py

```
from pyspark.sql import functions as F
```

```

from pyspark.sql.types import *

# Read from Kafka (or use socket for testing)
events_schema = StructType([
    StructField('user_id',    StringType(), True),
    StructField('event_type',  StringType(), True),
    StructField('amount',     DoubleType(), True),
    StructField('event_time',  TimestampType(), True),
])

stream = spark \
    .readStream \
    .format('kafka') \
    .option('kafka.bootstrap.servers', 'broker:9092') \
    .option('subscribe', 'transactions') \
    .load() \
    .select(F.from_json(F.col('value').cast('string'),
events_schema).alias('data')) \
    .select('data.*')

# Apply watermark for late data (allow up to 10 minutes late)
windowed_agg = stream \
    .withWatermark('event_time', '10 minutes') \
    .groupBy(
        F.window('event_time', '5 minutes'), # 5-min tumbling window
        'user_id'
    ).agg(
        F.count('*').alias('event_count'),
        F.sum('amount').alias('total_amount'),
        F.avg('amount').alias('avg_amount')
    )

# Write to Delta Lake
query = windowed_agg.writeStream \
    .format('delta') \
    .outputMode('append') \
    .option('checkpointLocation', '/checkpoints/transactions_agg') \
    .trigger(processingTime='30 seconds') \
    .start('/mnt/gold/delta/transactions_windowed')

query.awaitTermination()

```

4.6 Performance Tuning Guide

Practice Program 18: Performance Anti-Patterns and Fixes

PySpark — performance_tuning.py

```

# — Anti-Pattern 1: collect() on large DataFrame —————
# BAD: brings all data to driver (OOM risk)
# all_rows = large_df.collect() # DO NOT DO THIS

# GOOD: use take() for samples, or write to storage
sample = large_df.limit(100).collect() # safe
large_df.write.parquet('/output/path') # write result to storage

# — Anti-Pattern 2: Iterating rows with Python loop —————
# BAD: runs on driver, not distributed

```

```

# for row in df.collect(): process(row) # DO NOT DO THIS

# GOOD: use DataFrame transformations
df.withColumn('result', my_pandas_udf('col1', 'col2')).show()

# — Anti-Pattern 3: Too many small files —————
# Reading 10,000 small Parquet files is very slow
# GOOD: repartition before writing
df.repartition(200).write.parquet('/output/optimised')
# Or: coalesce for reducing only
df.coalesce(10).write.parquet('/output/small_result')

# — Anti-Pattern 4: shuffle.partitions too high/low —————
# Default 200 is too high for small data, too low for large data
# Rule of thumb: 128-256 MB per partition
total_bytes = 50 * 1024**3 # 50 GB
target_partition_bytes = 200 * 1024**2 # 200 MB
ideal_partitions = total_bytes // target_partition_bytes # 256
spark.conf.set('spark.sql.shuffle.partitions', ideal_partitions)

# — Anti-Pattern 5: Not using predicate pushdown —————
# GOOD: filter early
df_filtered = spark.read.parquet('/data') \
    .filter(F.col('year') == 2024) \
    .filter(F.col('region') == 'North')
# Parquet/Delta will only read relevant partitions+row-groups

# — Caching —————
# Cache when reusing a DataFrame in multiple actions
joined = orders.join(customers, 'customer_id')
joined.cache() # materialise on first action

count = joined.count()
agg = joined.groupBy('category').sum('amount')
top10 = joined.orderBy(F.desc('amount')).limit(10)

joined.unpersist() # free memory when done

# — AQE (Spark 3.0+) —————
spark.conf.set('spark.sql.adaptive.enabled', 'true')
spark.conf.set('spark.sql.adaptive.coalescePartitions.enabled', 'true')
spark.conf.set('spark.sql.adaptive.skewJoin.enabled', 'true')
# AQE automatically handles: coalescing small shuffle partitions,
# converting sort-merge join to broadcast, fixing data skew

```

SECTION 5: SNOWFLAKE — DEEP DIVE

5. Snowflake – In-Depth Guide

Snowflake is a cloud-native data platform that separates storage and compute at its architectural core. This separation means you pay only for the compute you use (Virtual Warehouses) and the storage you store, independently. Multiple teams can query the same data simultaneously with zero contention.

5.1 Core Architecture Components

Concept	Explanation
Cloud Services Layer	Always-on global brain: authentication, query optimisation, metadata management, transaction management. Billed at 10% of total compute usage typically.
Virtual Warehouse	Compute cluster (EC2/VMs) running the MPP query engine. Completely elastic: suspend/resume in seconds. Billed per-second.
Storage Layer	All table data stored as micro-partitions in cloud object storage (S3/Blob/GCS) in Snowflake's columnar encrypted format. Independent of compute.
Micro-partition	Auto-created contiguous storage unit (50-500MB uncompressed). Columnar within partition. Metadata: min/max, null counts, distinct counts per column.
Result Cache	24-hour cache of query results. Identical queries (same SQL + same data) return instantly with 0 credits used.

5.2 Virtual Warehouses — Full Guide

Practice Program 19: Warehouse Management

SQL — warehouse_management.sql

```
-- Create warehouses for different workloads
CREATE WAREHOUSE wh_etl
  WAREHOUSE_SIZE = 'LARGE'           -- 8 nodes
  AUTO_SUSPEND   = 120                -- suspend after 2 min idle
  AUTO_RESUME    = TRUE
  MIN_CLUSTER_COUNT = 1
  MAX_CLUSTER_COUNT = 3              -- multi-cluster for peak loads
  SCALING_POLICY = 'STANDARD'
  COMMENT        = 'ETL pipeline workloads';

CREATE WAREHOUSE wh_reporting
  WAREHOUSE_SIZE = 'X-SMALL'         -- 1 node — BI reports are read-only
  AUTO_SUSPEND   = 60
  AUTO_RESUME    = TRUE;
```

```
-- Monitor warehouse usage
SELECT  warehouse_name,
        SUM(credits_used)          AS total_credits,
        SUM(credits_used) * 3.0    AS approx_cost_usd,
        COUNT(*)                  AS query_count,
        AVG(execution_time)/1000 AS avg_exec_sec
FROM    SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
WHERE   start_time >= DATEADD('day', -30, CURRENT_TIMESTAMP())
GROUP BY warehouse_name
ORDER BY total_credits DESC;
```

5.3 Streams & Tasks — CDC Pipeline

Practice Program 20: Full Streams + Tasks CDC Pipeline

SQL — cdc_pipeline.sql

```
-- Source table
CREATE TABLE raw.orders (
  order_id      INT          NOT NULL,
  customer_id   INT,
  amount        NUMBER(10,2),
  status        VARCHAR(20),
  created_at    TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

-- Target table (Silver layer)
CREATE TABLE silver.orders (
  order_id      INT          NOT NULL PRIMARY KEY,
  customer_id   INT,
  amount        NUMBER(10,2),
  status        VARCHAR(20),
  created_at    TIMESTAMP_NTZ,
  last_updated  TIMESTAMP_NTZ,
  _action       VARCHAR(10)  -- INSERT / UPDATE / DELETE
);

-- Create stream to track changes
CREATE STREAM stream_raw_orders
  ON TABLE raw.orders
  APPEND_ONLY = FALSE      -- captures INSERT + UPDATE + DELETE
  SHOW_INITIAL_ROWS = FALSE; -- only new changes

-- Stored procedure to process stream
CREATE OR REPLACE PROCEDURE usp_process_orders_stream()
RETURNS STRING
LANGUAGE SQL
AS
$$
BEGIN
  -- Check if stream has data
  IF (SYSTEM$STREAM_HAS_DATA('stream_raw_orders')) THEN
    -- Process deletes
    DELETE FROM silver.orders tgt
    USING stream_raw_orders src
    WHERE src.order_id = tgt.order_id
      AND src.METADATA$ACTION = 'DELETE'
```



```

        AND src.METADATA$ISUPDATE = FALSE;

-- Process inserts and updates
MERGE INTO silver.orders tgt
USING (
    SELECT order_id, customer_id, amount, status, created_at,
           CURRENT_TIMESTAMP() AS last_updated,
           METADATA$ACTION AS _action
    FROM stream_raw_orders
    WHERE METADATA$ACTION = 'INSERT'
) src
ON tgt.order_id = src.order_id
WHEN MATCHED THEN UPDATE SET
    customer_id = src.customer_id,
    amount      = src.amount,
    status      = src.status,
    last_updated = src.last_updated,
    _action     = 'UPDATE'
WHEN NOT MATCHED THEN INSERT
    (order_id, customer_id, amount, status, created_at, last_updated,
     _action)
    VALUES (src.order_id, src.customer_id, src.amount, src.status,
            src.created_at, src.last_updated, 'INSERT');

    RETURN 'Stream processed successfully';
ELSE
    RETURN 'No new data in stream';
END IF;
END;
$$;

-- Create task to run every 5 minutes
CREATE TASK task_process_orders
    WAREHOUSE = wh_etl
    SCHEDULE  = '5 MINUTE'
    WHEN SYSTEM$STREAM_HAS_DATA('stream_raw_orders')
AS
    CALL usp_process_orders_stream();

-- Activate the task
ALTER TASK task_process_orders RESUME;

```

5.4 Snowflake with Python (Snowpark)

Practice Program 21: Full ETL Pipeline using Snowpark Python

Python — snowpark_pipeline.py

```

from snowflake.snowpark import Session
from snowflake.snowpark import functions as F
from snowflake.snowpark.types import *

# Connect
connection_params = {
    'account': 'myaccount.snowflakecomputing.com',
    'user': 'etl_user',
    'password': 'password', # use Secrets Manager in production
    'role': 'ETL_ROLE',

```

```

    'warehouse': 'WH_ETL',
    'database': 'PROD_DB',
    'schema': 'SILVER'
}
session = Session.builder.configs(connection_params).create()

# Read from Snowflake table as Snowpark DataFrame
orders_df = session.table('RAW.ORDERS')
customers_df = session.table('RAW.CUSTOMERS')

# Transformations (run inside Snowflake, no data movement)
clean_orders = orders_df \
    .filter(F.col('AMOUNT') > 0) \
    .filter(F.col('ORDER_ID').is_not_null()) \
    .withColumn('ORDER_DATE', F.to_date(F.col('ORDER_DATE_STR'), 'YYYY-MM-DD')) \
    .withColumn('YEAR', F.year(F.col('ORDER_DATE'))) \
    .withColumn('MONTH', F.month(F.col('ORDER_DATE'))) \
    .withColumnRenamed('CUST_ID', 'CUSTOMER_ID')

# Join with customers
enriched = clean_orders.join(
    customers_df.select('CUSTOMER_ID', 'CUSTOMER_NAME', 'SEGMENT', 'COUNTRY'),
    on='CUSTOMER_ID',
    how='left'
)

# Aggregation
summary = enriched.group_by('YEAR', 'MONTH', 'COUNTRY', 'SEGMENT') \
    .agg(
        F.count('ORDER_ID').alias('ORDER_COUNT'),
        F.sum('AMOUNT').alias('TOTAL_REVENUE'),
        F.avg('AMOUNT').alias('AVG_ORDER_VALUE'),
        F.count_distinct('CUSTOMER_ID').alias('UNIQUE_CUSTOMERS')
    ) \
    .withColumn('LOAD_TS', F.current_timestamp())

# Write to target table (overwrite or merge)
summary.write.mode('overwrite').save_as_table('GOLD.MONTHLY_SALES_SUMMARY')
print('Pipeline complete. Rows written:', summary.count())

# COPY INTO for bulk load from stage
session.sql('''
    COPY INTO RAW.EVENTS
    FROM @landing_stage/events/
    FILE FORMAT = (TYPE = 'PARQUET')
    MATCH_BY_COLUMN_NAME = CASE_INSENSITIVE
    PURGE = TRUE
''').collect()

session.close()

```

5.5 Advanced Snowflake — Semi-Structured Data

Practice Program 22: JSON & Array Operations

SQL — semi_structured.sql

```
-- Create table with VARIANT column for JSON
```

```

CREATE TABLE raw.events (event_id INT, payload VARIANT, loaded_at TIMESTAMP_NTZ);

-- Insert JSON
INSERT INTO raw.events
SELECT 1, PARSE_JSON('{
    "user_id": 101,
    "action": "purchase",
    "items": [{"sku":"A001","qty":2,"price":29.99},
{"sku":"B002","qty":1,"price":49.99}],
    "metadata": {"ip":"192.168.1.1","country":"IN","browser":"Chrome"}
}'), CURRENT_TIMESTAMP();

-- Query nested JSON with colon : operator
SELECT
    event_id,
    payload:user_id::INT           AS user_id,
    payload:action::STRING        AS action,
    payload:metadata:country::STRING AS country,
    payload:items[0]:sku::STRING  AS first_sku,
    payload:items[0]:price::FLOAT AS first_price,
    ARRAY_SIZE(payload:items)     AS item_count
FROM raw.events;

-- FLATTEN to explode array items into rows
SELECT
    e.event_id,
    e.payload:user_id::INT           AS user_id,
    item.value:sku::STRING           AS sku,
    item.value:qty::INT              AS qty,
    item.value:price::FLOAT          AS price,
    item.value:qty::INT * item.value:price::FLOAT AS line_total,
    item.index                      AS item_position
FROM raw.events e,
     LATERAL FLATTEN(input => e.payload:items) AS item;

-- Build structured table from JSON
CREATE TABLE silver.event_line_items AS
SELECT
    e.event_id,
    e.payload:user_id::INT           AS user_id,
    e.payload:action::STRING        AS action,
    e.payload:metadata:country::STRING AS country,
    item.value:sku::STRING           AS sku,
    item.value:qty::INT              AS qty,
    item.value:price::FLOAT          AS unit_price,
    item.value:qty::INT * item.value:price::FLOAT AS line_total,
    e.loaded_at
FROM raw.events e,
     LATERAL FLATTEN(input => e.payload:items) AS item;

```

SECTION 6: PYTHON FOR DATA ENGINEERING — DEEP DIVE

6. Python – Complete Data Engineering Guide

Python is the lingua franca of data engineering. It glues together every tool in the modern data stack — from querying databases and calling REST APIs to orchestrating Spark jobs and building data quality frameworks. This section covers Python concepts deeply with real data engineering examples.

6.1 Core Data Structures

Practice Program 23: Data Structures for Data Engineering

Python — data_structures.py

```
from collections import defaultdict, Counter, OrderedDict, deque
from dataclasses import dataclass, field
from typing import List, Dict, Optional

# — List: ordered, mutable —————
pipeline_steps = ['extract', 'validate', 'transform', 'load']

# List comprehension — filter only completed steps
completed = [s for s in pipeline_steps if s != 'load']

# Nested list comprehension — flatten 2D list
batches = [[1,2,3],[4,5,6],[7,8,9]]
flat = [item for batch in batches for item in batch]
print('Flat:', flat)  # [1, 2, 3, 4, 5, 6, 7, 8, 9]

# — Dict: key-value store O(1) average lookup —————
schema = {'id': 'int', 'name': 'str', 'amount': 'float', 'date': 'date'}

# Dict comprehension — invert schema (value: type -> type: [values])
by_type = defaultdict(list)
for col, dtype in schema.items():
    by_type[dtype].append(col)
print(dict(by_type))  # {'int':['id'], 'str':['name'], ...}

# — Counter: count occurrences —————
raw_events = ['login','purchase','login','view','purchase','purchase','login']
c = Counter(raw_events)
print(c.most_common(2))  # [('login', 3), ('purchase', 3)]

# — Deque: efficient queue (O(1) append/popleft) —————
processing_queue = deque(maxlen=1000)  # circular buffer
for i in range(5): processing_queue.append(f'record_{i}')
record = processing_queue.popleft()  # FIFO
print('Processed:', record)

# — Dataclass: structured record with validation —————
```

```

@dataclass
class TableConfig:
    name: str
    schema: str
    primary_key: str
    watermark_col: Optional[str] = None
    batch_size: int = 100_000
    partitions: List[str] = field(default_factory=list)

    def __post_init__(self):
        if not self.name:
            raise ValueError('Table name cannot be empty')
        if self.batch_size <= 0:
            raise ValueError('batch_size must be positive')

# Usage
config = TableConfig(
    name='orders', schema='dbo', primary_key='order_id',
    watermark_col='updated_at', batch_size=50_000,
    partitions=['year', 'month']
)
print(config)

```

6.2 Generators & Iterators

Practice Program 24: Memory-Efficient File Processing

Python — generators.py

```

import csv
import json
from typing import Iterator, Generator

# — Generator for large CSV processing —————
def read_csv_chunks(filepath: str, chunk_size: int = 10_000) -> Generator:
    '''Read CSV in chunks without loading entire file into memory'''
    with open(filepath, 'r', encoding='utf-8') as f:
        reader = csv.DictReader(f)
        chunk = []
        for row in reader:
            chunk.append(row)
            if len(chunk) >= chunk_size:
                yield chunk
                chunk = []
        if chunk: # yield remaining rows
            yield chunk

# Usage — process 10M row CSV with constant memory
total_rows = 0
for chunk in read_csv_chunks('large_file.csv', chunk_size=50_000):
    # process each chunk
    processed = [{'id': r['id'], 'amount': float(r['amount'])} for r in chunk
                  if float(r.get('amount', 0)) > 0]
    total_rows += len(processed)
    # write processed chunk to database
print(f'Processed {total_rows:,} valid rows')

# — Generator pipeline — chain transformations lazily —

```

```

def extract_json_lines(filepath: str) -> Iterator[dict]:
    with open(filepath) as f:
        for line in f:
            try:
                yield json.loads(line.strip())
            except json.JSONDecodeError:
                continue # skip malformed lines

def filter_events(records: Iterator, event_type: str) -> Iterator:
    for r in records:
        if r.get('type') == event_type:
            yield r

def enrich_events(records: Iterator) -> Iterator:
    for r in records:
        r['amount_usd'] = round(r.get('amount', 0) * 0.012, 2)
        r['processed'] = True
        yield r

def batch_records(records: Iterator, size: int) -> Iterator[list]:
    batch = []
    for r in records:
        batch.append(r)
        if len(batch) >= size:
            yield batch
            batch = []
    if batch: yield batch

# Compose the pipeline - lazy, no intermediate lists
pipeline = batch_records(
    enrich_events(
        filter_events(
            extract_json_lines('events.jsonl'),
            event_type='purchase'
        )
    ),
    size=500
)

for batch in pipeline:
    # db.bulk_insert(batch)
    print(f'Writing batch of {len(batch)} records')

```

6.3 Decorators for Data Engineering

Practice Program 25: Retry, Timer, and Logging Decorators

Python — decorators.py

```

import functools
import time
import logging
import random
from typing import Callable, Any, Type, Tuple

logger = logging.getLogger(__name__)

# — Timer decorator —————

```

```

def timer(func: Callable) -> Callable:
    @functools.wraps(func) # preserves original func metadata
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        try:
            result = func(*args, **kwargs)
            elapsed = time.perf_counter() - start
            logger.info(f'{func.__name__} completed in {elapsed:.3f}s')
            return result
        except Exception as e:
            elapsed = time.perf_counter() - start
            logger.error(f'{func.__name__} FAILED after {elapsed:.3f}s: {e}')
            raise
    return wrapper

# — Retry decorator with exponential backoff —
def retry(
    max_attempts: int = 3,
    delay: float = 1.0,
    backoff: float = 2.0,
    exceptions: Tuple[Type[Exception], ...] = (Exception,)
):
    def decorator(func: Callable) -> Callable:
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            attempt = 0
            current_delay = delay
            while attempt < max_attempts:
                try:
                    return func(*args, **kwargs)
                except exceptions as e:
                    attempt += 1
                    if attempt == max_attempts:
                        logger.error(f'{func.__name__} failed after {max_attempts}
attempts')
                        raise
                    jitter = random.uniform(0, 0.3 * current_delay)
                    sleep_time = current_delay + jitter
                    logger.warning(f'{func.__name__} attempt {attempt} failed: {e}.
Retrying in {sleep_time:.1f}s')
                    time.sleep(sleep_time)
                    current_delay *= backoff
            return wrapper
        return decorator

# — Validate schema decorator —
def validate_dataframe(required_cols: list, min_rows: int = 1):
    def decorator(func: Callable) -> Callable:
        @functools.wraps(func)
        def wrapper(df, *args, **kwargs):
            import pandas as pd
            if not isinstance(df, pd.DataFrame):
                raise TypeError(f'Expected DataFrame, got {type(df)}')
            missing = set(required_cols) - set(df.columns)
            if missing:
                raise ValueError(f'Missing columns: {missing}')
            if len(df) < min_rows:
                raise ValueError(f'DataFrame has {len(df)} rows, need at least
{min_rows}')
            return func(df, *args, **kwargs)
        return wrapper

```

```

        return decorator

# — Usage —————
@timer
@retry(max_attempts=3, delay=2.0, exceptions=(ConnectionError, TimeoutError))
def call_external_api(endpoint: str) -> dict:
    # Simulate API call
    import requests
    response = requests.get(endpoint, timeout=10)
    response.raise_for_status()
    return response.json()

@validate_dataframe(required_cols=['order_id', 'amount', 'customer_id'])
def process_orders(df):
    return df[df['amount'] > 0].copy()

```

6.4 Context Managers

Practice Program 26: Database Connection Pool Context Manager

Python — context_managers.py

```

from contextlib import contextmanager, asynccontextmanager
import time
import logging

logger = logging.getLogger(__name__)

# — Class-based context manager —————
class DatabaseConnection:
    def __init__(self, connection_string: str, autocommit: bool = False):
        self.conn_str = connection_string
        self.autocommit = autocommit
        self.conn = None
        self.cursor = None
        self._start_time = None

    def __enter__(self):
        import pyodbc
        logger.info(f'Opening DB connection to {self.conn_str[:30]}...')
        self._start_time = time.perf_counter()
        self.conn = pyodbc.connect(self.conn_str)
        self.conn.autocommit = self.autocommit
        self.cursor = self.conn.cursor()
        return self.cursor

    def __exit__(self, exc_type, exc_val, exc_tb):
        elapsed = time.perf_counter() - self._start_time
        if exc_type is None:
            if not self.autocommit:
                self.conn.commit()
            logger.info(f'Transaction committed ({elapsed:.2f}s)')
        else:
            self.conn.rollback()
            logger.error(f'Transaction rolled back due to {exc_type.__name__}: {exc_val}')
        self.cursor.close()
        self.conn.close()

```



```

        return False # re-raise exceptions

# — Generator-based context manager —————
@contextmanager
def temp_directory(prefix: str = 'etl_'):
    import tempfile, shutil
    tmpdir = tempfile.mkdtemp(prefix=prefix)
    logger.debug(f'Created temp dir: {tmpdir}')
    try:
        yield tmpdir
    finally:
        shutil.rmtree(tmpdir, ignore_errors=True)
        logger.debug(f'Cleaned up temp dir: {tmpdir}')

@contextmanager
def pipeline_run(pipeline_name: str):
    start = time.perf_counter()
    logger.info(f'[START] Pipeline: {pipeline_name}')
    try:
        yield
        elapsed = time.perf_counter() - start
        logger.info(f'[SUCCESS] Pipeline: {pipeline_name} completed in {elapsed:.2f}s')
    except Exception as e:
        elapsed = time.perf_counter() - start
        logger.error(f'[FAILED] Pipeline: {pipeline_name} failed after {elapsed:.2f}s: {e}')
        raise

# — Usage —————
with pipeline_run('daily_etl'):
    with temp_directory('etl_staging_') as tmpdir:
        # download files to tmpdir
        print(f'Working in {tmpdir}')
        # temp files cleaned up automatically

# DB usage
conn_str = 'DRIVER={SQL Server};SERVER=localhost;DATABASE=mydb'
with DatabaseConnection(conn_str) as cursor:
    cursor.execute('SELECT COUNT(*) FROM dbo.orders')
    count = cursor.fetchone()[0]
    print(f'Orders: {count}')
# connection committed and closed automatically

```

6.5 Pandas — Advanced Data Engineering

Practice Program 27: Complete Pandas ETL

Python — pandas_etl.py

```

import pandas as pd
import numpy as np
from pathlib import Path

# — Read and validate —————
def load_and_validate(filepath: str) -> pd.DataFrame:
    df = pd.read_csv(
        filepath,

```

```

        dtype={
            'order_id': 'Int64',      # nullable integer
            'customer_id': 'Int64',
            'product_sku': 'string',  # nullable string
            'quantity': 'Int32',
        },
        parse_dates=['order_date'],
        date_format='%Y-%m-%d',
    )
    # Validate required columns
    required = {'order_id', 'customer_id', 'amount', 'order_date'}
    missing = required - set(df.columns)
    if missing: raise ValueError(f'Missing: {missing}')
    # Validate no duplicate order IDs
    dupes = df[df.duplicated('order_id', keep=False)]
    if len(dupes) > 0:
        print(f'WARNING: {len(dupes)} duplicate order_ids found')
        df = df.drop_duplicates('order_id', keep='last')
    return df

# ——— Clean and transform ———
def transform(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()

    # Handle nulls
    df['amount'] = pd.to_numeric(df['amount'], errors='coerce')
    df = df[df['amount'].notna() & (df['amount'] > 0)]
    df['region'] = df['region'].fillna('UNKNOWN')

    # Date features
    df['year'] = df['order_date'].dt.year
    df['month'] = df['order_date'].dt.month
    df['quarter'] = df['order_date'].dt.quarter
    df['weekday'] = df['order_date'].dt.day_name()
    df['is_weekend'] = df['order_date'].dt.dayofweek >= 5

    # Binning
    df['amount_tier'] = pd.cut(
        df['amount'],
        bins=[0, 100, 500, 1000, float('inf')],
        labels=['Small', 'Medium', 'Large', 'Enterprise'],
        right=True
    )

    # Normalise
    df['amount_norm'] = (df['amount'] - df['amount'].mean()) / df['amount'].std()

    # String cleaning
    df['region'] = df['region'].str.upper().str.strip()

    return df

# ——— Aggregate ———
def build_summary(df: pd.DataFrame) -> pd.DataFrame:
    summary = df.groupby(['year', 'month', 'region']).agg(
        order_count = ('order_id', 'count'),
        total_revenue = ('amount', 'sum'),
        avg_order_value = ('amount', 'mean'),
        median_order_value = ('amount', 'median'),
        max_order = ('amount', 'max'),
    )

```

```

        unique_customers      = ('customer_id', 'nunique'),
        orders_per_customer    = ('order_id',    lambda x: len(x) /
df.loc[x.index, 'customer_id'].nunique()),
    ).round(2).reset_index()

    # Running total per region
    summary = summary.sort_values(['region', 'year', 'month'])
    summary['running_revenue'] =
summary.groupby('region')['total_revenue'].cumsum()

    # Revenue rank within month
    summary['revenue_rank'] = summary.groupby(['year', 'month'])['total_revenue'] \
        .rank(method='dense', ascending=False).astype(int)

    return summary

# — Pivot for report —————
def build_pivot(df: pd.DataFrame) -> pd.DataFrame:
    pivot = df.pivot_table(
        values='amount',
        index=['year', 'month'],
        columns='region',
        aggfunc='sum',
        fill_value=0
    )
    pivot.columns = [f'revenue_{c.lower()}' for c in pivot.columns]
    pivot['grand_total'] = pivot.sum(axis=1)
    return pivot.reset_index()

# — Pipeline runner —————
if __name__ == '__main__':
    raw_df = load_and_validate('orders.csv')
    clean_df = transform(raw_df)
    summ_df = build_summary(clean_df)
    pivot_df = build_pivot(clean_df)

    print('Summary shape:', summ_df.shape)
    print(summ_df.head())
    print('Pivot:')
    print(pivot_df.head())

    # Write outputs
    summ_df.to_parquet('summary.parquet', index=False)
    pivot_df.to_csv('pivot_report.csv', index=False)

```

6.6 Azure SDK — ADLS & Blob Storage

Practice Program 28: ADLS Operations Helper Class

Python — adls_helper.py

```

from azure.storage.filedatalake import DataLakeServiceClient
from azure.identity import DefaultAzureCredential
from pathlib import PurePosixPath
import io, json, logging
from typing import List, Optional, Iterator

class ADLSHelper:

```

```

'''Helper class for ADLS Gen2 operations using Managed Identity'''

def __init__(self, account_name: str):
    self.account_name = account_name
    url = f'https://{account_name}.dfs.core.windows.net'
    self.client = DataLakeServiceClient(
        account_url=url,
        credential=DefaultAzureCredential()
    )
    self.logger = logging.getLogger(self.__class__.__name__)

def list_files(self, container: str, path: str = '',
               pattern: str = None) -> List[str]:
    '''List all files in a path, optionally filtered by suffix'''
    fs_client = self.client.get_file_system_client(container)
    paths = fs_client.get_paths(path=path, recursive=True)
    files = [p.name for p in paths if not p.is_directory]
    if pattern:
        files = [f for f in files if f.endswith(pattern)]
    return files

def read_json(self, container: str, path: str) -> dict:
    '''Read a JSON file from ADLS and return as dict'''
    fs_client = self.client.get_file_system_client(container)
    file_client = fs_client.get_file_client(path)
    download = file_client.download_file()
    data = download.readall()
    return json.loads(data.decode('utf-8'))

def write_json(self, container: str, path: str, data: dict,
               overwrite: bool = True) -> None:
    '''Write a dict as JSON to ADLS'''
    content = json.dumps(data, default=str, indent=2).encode('utf-8')
    fs_client = self.client.get_file_system_client(container)
    file_client = fs_client.get_file_client(path)
    file_client.upload_data(content, overwrite=overwrite, length=len(content))
    self.logger.info(f'Written {len(content)} bytes to {container}/{path}')

def copy_file(self, src_container: str, src_path: str,
               dst_container: str, dst_path: str) -> None:
    '''Copy a file from one location to another'''
    import io
    src_fs = self.client.get_file_system_client(src_container)
    src_file = src_fs.get_file_client(src_path)
    data = src_file.download_file().readall()

    dst_fs = self.client.get_file_system_client(dst_container)
    dst_file = dst_fs.get_file_client(dst_path)
    dst_file.upload_data(data, overwrite=True, length=len(data))
    self.logger.info(f'Copied {src_container}/{src_path} ->
{dst_container}/{dst_path}')

def move_to_archive(self, container: str, path: str) -> None:
    '''Archive a file by moving it to an archive prefix'''
    archive_path = str(PurePosixPath('archive') / path)
    self.copy_file(container, path, container, archive_path)
    fs = self.client.get_file_system_client(container)
    fs.get_file_client(path).delete_file()

```

Usage

```

if __name__ == '__main__':
    adls = ADLSHelper('myadlsaccount')

    # List all Parquet files in bronze container
    parquet_files = adls.list_files('bronze', 'sales/2024/', '.parquet')
    print(f'Found {len(parquet_files)} Parquet files')

    # Read config
    config = adls.read_json('configs', 'etl/pipeline_config.json')

    # Write result metadata
    metadata = {'run_date': '2024-01-15', 'rows_processed': 150000}
    adls.write_json('metadata', 'runs/2024-01-15.json', metadata)

```

6.7 Data Quality Framework

Practice Program 29: Reusable Data Quality Check Engine

Python — data_quality.py

```

from dataclasses import dataclass, field
from typing import Callable, List, Dict, Any, Optional
from enum import Enum
import pandas as pd
import json
from datetime import datetime

class Severity(Enum):
    ERROR = 'ERROR'
    WARNING = 'WARNING'
    INFO = 'INFO'

@dataclass
class CheckResult:
    check_name: str
    passed: bool
    severity: Severity
    message: str
    rows_affected: int = 0
    details: Optional[Dict] = None

@dataclass
class DataQualityReport:
    table: str
    run_time: str = field(default_factory=lambda: datetime.utcnow().isoformat())
    results: List[CheckResult] = field(default_factory=list)

    @property
    def passed(self) -> bool:
        return all(r.passed for r in self.results if r.severity == Severity.ERROR)

    @property
    def summary(self) -> Dict:
        return {
            'table': self.table,
            'run_time': self.run_time,
            'total': len(self.results),
            'passed': sum(1 for r in self.results if r.passed),

```

```

        'failed':    sum(1 for r in self.results if not r.passed),
        'errors':    [r for r in self.results if not r.passed and r.severity ==
Severity.ERROR],
    }

class DataQualityEngine:
    def __init__(self, df: pd.DataFrame, table_name: str):
        self.df = df
        self.report = DataQualityReport(table=table_name)

    def check_not_null(self, columns: List[str], severity=Severity.ERROR):
        for col in columns:
            null_count = self.df[col].isna().sum()
            self.report.results.append(CheckResult(
                check_name = f'not_null:{col}',
                passed      = null_count == 0,
                severity    = severity,
                message     = f'{col} has {null_count} null values' if null_count
else f'{col} has no nulls',
                rows_affected = int(null_count)
            ))
        return self

    def check_unique(self, columns: List[str], severity=Severity.ERROR):
        for col in columns:
            dup_count = self.df.duplicated(col).sum()
            self.report.results.append(CheckResult(
                check_name = f'unique:{col}',
                passed      = dup_count == 0,
                severity    = severity,
                message     = f'{col} has {dup_count} duplicates' if dup_count else
f'{col} is unique',
                rows_affected = int(dup_count)
            ))
        return self

    def check_range(self, column: str, min_val, max_val,
severity=Severity.WARNING):
        out_of_range = ((self.df[column] < min_val) | (self.df[column] >
max_val)).sum()
        self.report.results.append(CheckResult(
            check_name     = f'range:{column}',
            passed         = out_of_range == 0,
            severity       = severity,
            message        = f'{column}: {out_of_range} rows outside
[{min_val},{max_val}]',
            rows_affected  = int(out_of_range),
            details        = {'min_val': min_val, 'max_val': max_val}
        ))
        return self

    def check_row_count(self, min_rows: int, max_rows: int = None,
severity=Severity.ERROR):
        count = len(self.df)
        passed = count >= min_rows and (max_rows is None or count <= max_rows)
        self.report.results.append(CheckResult(
            check_name = 'row_count',
            passed     = passed,
            severity   = severity,
            message    = f'Row count {count} (expected {min_rows}-{max_rows or
"inf"})',

```

```

        rows_affected = count
    ))
    return self

    def check_referential_integrity(self, column: str, valid_values: set,
severity=Severity.ERROR):
        invalid = ~self.df[column].isin(valid_values)
        count = invalid.sum()
        self.report.results.append(CheckResult(
            check_name      = f'ref_integrity:{column}',
            passed          = count == 0,
            severity        = severity,
            message         = f'{column}: {count} rows with invalid values',
            rows_affected   = int(count),
            details         = {'invalid_values': list(self.df.loc[invalid,
column].unique()[:5])})
        ))
    return self

    def run(self) -> DataQualityReport:
        return self.report

# — Usage —————
df = pd.DataFrame({
    'order_id':    [1, 2, 2, 3, 4],
    'customer_id': [101, 102, 102, None, 105],
    'amount':      [500, -10, 300, 200, 150000],
    'status':      ['COMPLETED', 'SHIPPED', 'COMPLETED', 'PROCESSING', 'INVALID'],
})

report = (
    DataQualityEngine(df, 'orders')
    .check_not_null(['order_id', 'customer_id', 'amount'])
    .check_unique(['order_id'])
    .check_range('amount', min_val=0, max_val=100000)
    .check_row_count(min_rows=1)
    .check_referential_integrity('status',
{'COMPLETED', 'SHIPPED', 'PROCESSING', 'CANCELLED'})
    .run()
)

for result in report.results:
    status = 'PASS' if result.passed else 'FAIL'
    print(f'[{status}] [{result.severity.value}] {result.check_name}:
{result.message}')

print('Overall passed:', report.passed)

```

6.8 Complete End-to-End ETL Practice Program

Practice Program 30: Production-Grade ETL with Error Handling, Logging & Metrics

Python — production_etl.py

```

#!/usr/bin/env python3
"""
Production ETL: Extract from SQL Server -> Validate -> Transform -> Load to
Snowflake

```

```

Features: retry, logging, metrics, checkpointing, alerting
"""
import logging, json, time, sys
from datetime import datetime, timedelta
from pathlib import Path
from dataclasses import dataclass, field, asdict
from typing import Optional

import pandas as pd
import pyodbc
import snowflake.connector
from snowflake.connector.pandas_tools import write_pandas

# --- Logging setup ---
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s | %(levelname)-8s | %(name)s | %(message)s',
    handlers=[
        logging.StreamHandler(sys.stdout),
        logging.FileHandler(f'etl_{datetime.now():%Y%m%d}.log')
    ]
)
log = logging.getLogger('etl.main')

# --- Config ---
@dataclass
class ETLConfig:
    source_server: str
    source_database: str
    source_table: str
    watermark_col: str
    sf_account: str
    sf_database: str
    sf_schema: str
    sf_table: str
    sf_warehouse: str
    batch_size: int = 100_000
    retry_attempts: int = 3

@dataclass
class RunMetrics:
    run_id: str = field(default_factory=lambda: f'run_{int(time.time())}')
    start_time: str = ''
    end_time: str = ''
    rows_extracted: int = 0
    rows_valid: int = 0
    rows_rejected: int = 0
    rows_loaded: int = 0
    status: str = 'RUNNING'
    error_message: str = ''

    def elapsed_seconds(self) -> float:
        if self.start_time and self.end_time:
            fmt = '%Y-%m-%dT%H:%M:%S.%f'
            return (datetime.fromisoformat(self.end_time) -
                    datetime.fromisoformat(self.start_time)).total_seconds()
        return 0

# --- Extract ---
def extract(config: ETLConfig, watermark: str) -> pd.DataFrame:

```



```

log.info(f'Extracting {config.source_table} since {watermark}')
conn_str = (
    f'DRIVER={{ODBC Driver 18 for SQL Server}};'
    f'SERVER={config.source_server};DATABASE={config.source_database};'
    'Trusted_Connection=yes;'
)
query = f'''
    SELECT TOP {config.batch_size} *
    FROM    [{config.source_table}]
    WHERE   {config.watermark_col} > '{watermark}'
    ORDER BY {config.watermark_col}
'''
with pyodbc.connect(conn_str) as conn:
    df = pd.read_sql(query, conn)
log.info(f'Extracted {len(df):,} rows')
return df

# ——— Validate ———
def validate(df: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
    log.info('Validating...')
    valid_mask = (
        df['order_id'].notna() &
        df['amount'].gt(0) &
        df['customer_id'].notna()
    )
    valid = df[valid_mask].copy()
    invalid = df[~valid_mask].copy()
    log.info(f'Valid: {len(valid):,} | Rejected: {len(invalid):,}')
    return valid, invalid

# ——— Transform ———
def transform(df: pd.DataFrame) -> pd.DataFrame:
    log.info('Transforming...')
    df = df.copy()
    df.columns = [c.upper() for c in df.columns]
    df['ORDER_DATE'] = pd.to_datetime(df['ORDER_DATE'])
    df['YEAR'] = df['ORDER_DATE'].dt.year
    df['MONTH'] = df['ORDER_DATE'].dt.month
    df['AMOUNT'] = df['AMOUNT'].round(2)
    df['ETL_LOAD_TS'] = pd.Timestamp.utcnow()
    return df

# ——— Load ———
def load_to_snowflake(df: pd.DataFrame, config: ETLConfig) -> int:
    log.info(f'Loading {len(df):,} rows to {config.sf_table}')
    conn = snowflake.connector.connect(
        account=config.sf_account, user='etl_user', password='password',
        warehouse=config.sf_warehouse, database=config.sf_database,
        schema=config.sf_schema
    )
    success, _, nrows, _ = write_pandas(
        conn, df, config.sf_table.upper(),
        auto_create_table=True, overwrite=False
    )
    conn.close()
    if not success: raise RuntimeError('Snowflake load failed')
    log.info(f'Loaded {nrows:,} rows')
    return nrows

# ——— Main orchestrator ———
def run_pipeline(config: ETLConfig, watermark: str) -> RunMetrics:

```

```

metrics = RunMetrics(start_time=datetime.utcnow().isoformat())
try:
    raw_df = extract(config, watermark)
    metrics.rows_extracted = len(raw_df)
    if raw_df.empty:
        log.info('No new data. Exiting.')
        metrics.status = 'NO_DATA'
        return metrics
    valid_df, bad_df = validate(raw_df)
    metrics.rows_valid = len(valid_df)
    metrics.rows_rejected = len(bad_df)
    final_df = transform(valid_df)
    metrics.rows_loaded = load_to_snowflake(final_df, config)
    metrics.status = 'SUCCESS'
except Exception as e:
    metrics.status = 'FAILED'
    metrics.error_message = str(e)
    log.exception('Pipeline failed')
finally:
    metrics.end_time = datetime.utcnow().isoformat()
    log.info(f'Run complete: {json.dumps(asdict(metrics), default=str,
indent=2)}')
    return metrics

if __name__ == '__main__':
    cfg = ETLConfig(
        source_server='sql-server-prod', source_database='SalesDB',
        source_table='dbo.Orders', watermark_col='UpdatedAt',
        sf_account='myaccount.snowflakecomputing.com',
        sf_database='PROD_DB', sf_schema='SILVER', sf_table='ORDERS',
        sf_warehouse='WH_ETL'
    )
    watermark = '2024-01-01'
    result = run_pipeline(cfg, watermark)
    sys.exit(0 if result.status == 'SUCCESS' else 1)

```

SECTION 7: PRACTICE PROGRAMS SUMMARY & TIPS

7. Practice Programs Summary

This document contains 30 complete practice programs. Here is the full index for quick reference:

Concept	Explanation
PP 1	ADF — Copy CSV to Parquet (Pipeline JSON definition)
PP 2	ADF — ForEach + Execute Pipeline pattern
PP 3	ADF — Data Flow expression: SCD Type 1 upsert logic
PP 4	ADF — Full Incremental Pipeline triggered from Python SDK
PP 5	ADF — Generate ADF Pipeline Config dynamically in Python
PP 6	Synapse — Serverless SQL: OPENROWSET + External Table
PP 7	Synapse — Star Schema build with CTAS and statistics
PP 8	Synapse — Python ODBC connector with AAD token
PP 9	Databricks — Full Delta Lake Operations (create, merge, time travel, optimize, vacuum)
PP 10	Databricks — SCD Type 2 with Delta MERGE
PP 11	Databricks — Autoloader Streaming Ingestion to Delta
PP 12	Databricks — Bronze → Silver → Gold Medallion Pipeline
PP 13	Databricks — Unity Catalog DDL + column masking
PP 14	PySpark — All Essential DataFrame Operations (filter, agg, window)
PP 15	PySpark — All Join Types (inner, left, anti, semi, broadcast, non-equi)
PP 16	PySpark — Regular UDF + Pandas UDF (Haversine example)
PP 17	PySpark — Structured Streaming with Watermark + Kafka
PP 18	PySpark — Performance Tuning (anti-patterns + AQE config)
PP 19	Snowflake — Warehouse management SQL
PP 20	Snowflake — Full Streams + Tasks CDC pipeline with MERGE
PP 21	Snowflake — Snowpark Python ETL pipeline
PP 22	Snowflake — Semi-structured JSON / FLATTEN / LATERAL
PP 23	Python — Data Structures for data engineering
PP 24	Python — Memory-efficient generators & lazy pipeline
PP 25	Python — Retry, Timer, and Schema-validate decorators

PP 26	Python — Context managers for DB and temp files
PP 27	Python — Complete Pandas ETL with pivot and summary
PP 28	Python — ADLS Gen2 helper class with Managed Identity
PP 29	Python — Reusable Data Quality Check Engine
PP 30	Python — Production ETL: SQL → Validate → Transform → Snowflake

Key Interview Tips

- Always explain WHY you chose a design (hash distribution vs replicate, broadcast join vs sort-merge).
- Know the difference between LAZY vs EAGER evaluation — Spark is lazy, use explain() to inspect plans.
- Demonstrate Delta Lake knowledge: ACID, time travel, MERGE, OPTIMIZE/ZORDER, VACUUM.
- For ADF, know the incremental load watermark pattern and CI/CD with ARM templates + Azure DevOps.
- For Snowflake, understand the three-layer architecture, micro-partitions, and Zero-Copy Clone.
- For Python, emphasise generators (memory), decorators (retry/logging), and context managers (resources).
- Mention performance tuning: avoid data movement (Synapse), reduce shuffles (Spark), auto-suspend (Snowflake).
- Show awareness of cost: DIU in ADF, DWU scaling in Synapse, credit consumption in Snowflake, cluster type in Databricks.

Best of Luck in Your Interviews!